PROGRAMACIÓN ORIENTADA A OBJETOS

ALUMNOS:

Astrit Airan Cetzal Cetzal

Venus Getsemaní Semino Aleman

Euruviel Marquez Martines

DOCENTE: Cinthia Carina Castillo

Fecha: 26/01/2026

GRUPO:2

Actividad: Proyecto Final Unidad I

Descripción del sistema:

Un estacionamiento debe llevar el registro de entrada y salidas de los vehículos que ingresen al lugar. Todos los vehículos que ingresan ocupan un espacio, y el sistema debe poder coordinar la asignación de espacio libre, al igual que liberar un espacio y consultar cuantas entradas hubo en general.

Clase: Espacio

Responsabilidad: Representar un lugar físico y su ocupación en el estacionamiento.

Atributo	Tipo	Por qué existe
numero	int	Identificar el lugar del estacionamiento
estado	String	Saber si el lugar está libre o ocupado
vehiculoActual	Vehiculo	Conocer el tipo del vehículo

Método	Tipo	Por qué existe
ocupar(...)	Metodo	Establecer cuál vehículo está ocupando el lugar
liberar()	Metodo	Conocer cuando un espacio está ocupado

estaLibre()	Metodo	Identificar que el espacio esté libre
Espacio(...)	Constructor parametrizado	Crear el espacio con los datos del cliente
getNumero	Método Getter	Obtener el número del espacio

Clase: SistemaEstacionamiento

Responsabilidad: coordinar la asignación (entradas) y liberación (salidas) de espacios y tickets.

Atributos	Tipo	Por qué existe
nombre	String	Identificar al sistema
totalEntradas	static int	Contar cuántos coches entraron en el día
totalSalidas	static int	Cuenta cuántos coches se fueron al terminar el día

Método	Tipo	Por qué existe

registrarEntrada(...)	Ticket	registra la hora en al que el vehículo entró al estacionamiento
registrarSalidas(...)	Método	registra la hora de salida del vehículo del estacionamiento
getTotalEntradas()	Método static Getter int	Consultar el número de entradas
getTotalSalidas()	Método static Getter int	Consultar el número de de salidas
SistemaEstacionamiento()	Constructor parametrizado	Crea los datos iniciales del sistema del estacionamiento

Clase: Ticket

Responsabilidad: Representa el registro de entrada/salida asociado a un vehículo.

Atributo	Tipo	Porque existe
folio	String	Identifica al ticket
horaEntrada	String	Indica la hora que se estacionó un vehículo

horaSalida	String	Indica la hora que se salió un vehículo

Método	Tipo	Por qué existe
finalizar(...)	Método	Mostrar la hora de la salida
obtenerDetalle()	Método	Consultar los detalles
entregado()	Método	actualizar si estacionamiento está abierto o cerrado
Ticket(...)	Constructor parametrizado	Crear al ticket con datos iniciales

clase: Vehiculo

Responsabilidad: representa los diferentes vehículos que ingresan al estacionamiento

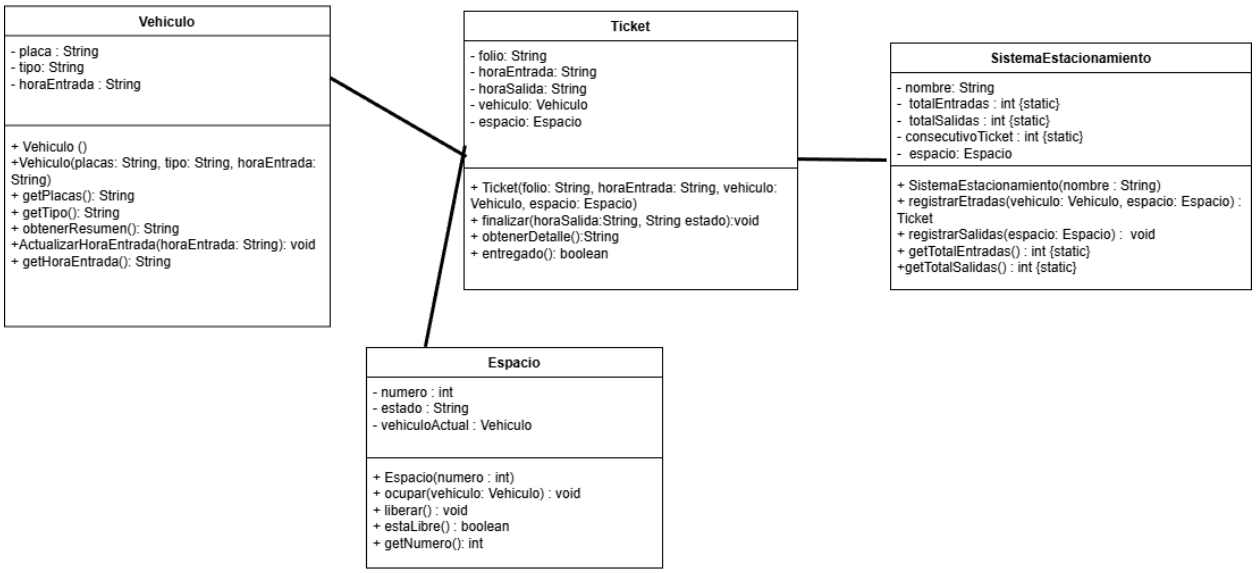
Atributo	Tipo	Porque existe
placa	String	para identificar el vehiculo
tipo	String	para saber que tipo(la marca o modelo) del auto

horaDeEntrada	String	indica la hora en la que entró el vehículo

Método	Tipo	Por qué existe
getPlacas()	Método Getter	para consultar los datos de la placa
obtenerResumen()	Método	sirve para obtener la información de forma resumida
ActualizarHoraEntrada()	Método	para actualizar la información
Vehiculo(...)	Constructor parametrizado	para crear un vehículo con información proporcionada
Vehiculo	Constructor default	Permite crear al objeto con valores iniciales
getTipo	Método Getter	Permite consultar el tipo de vehículo registrado

UML

https://app.diagrams.net/#G1Rx6vbslSnicMrJ7rUxsFW49k9ouGglf-#%7B%22pageId%22%3A%22mbHwUQnA12hQRh79CzIH%22%7D

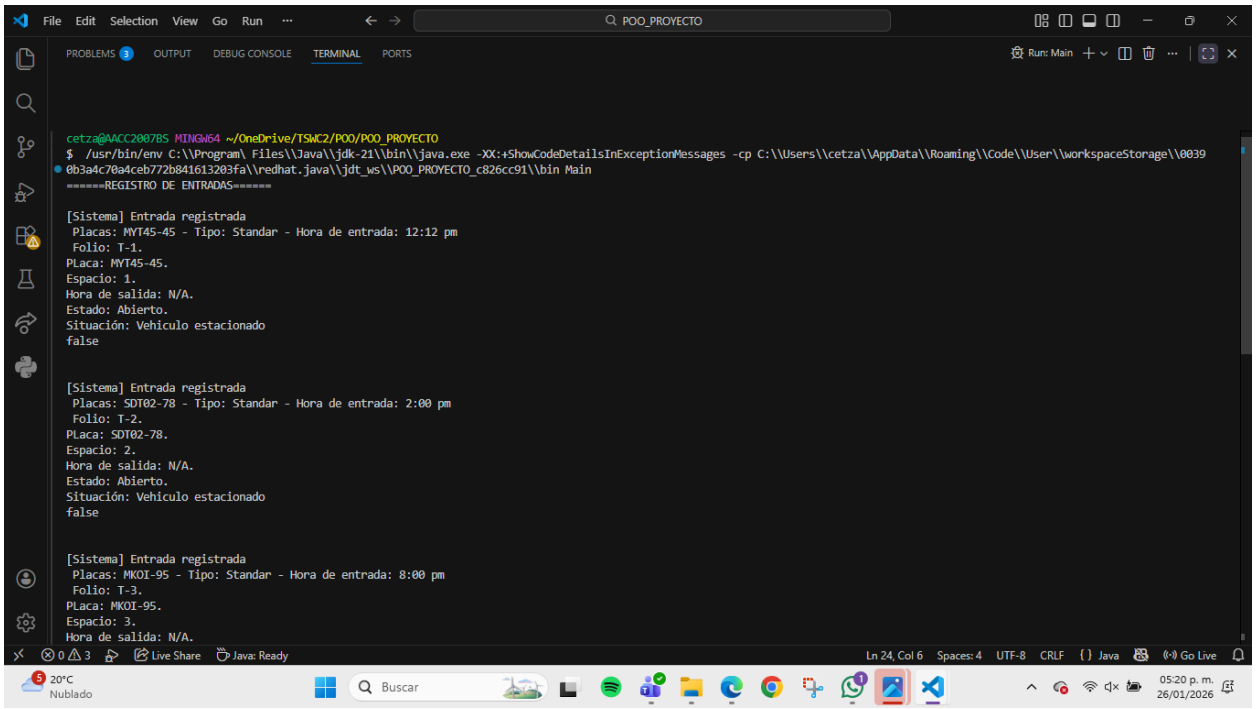


MENSAJES

1. SistemaEstacionamineto-> ocupar(vehiculo) -> Espacio
2. SistemaEstacionamineto-> getHoraEntrada()-> Vehiculo
3. SistemaEstacionamineto-> Ticket() -> Ticket
4. Ticket-> getPlacas() -> Vehiculo
5. Ticket-> getNumero() -> Espacio
6. Ticket-> liberar() -> Espacio
7. Ticket-> estaLibre -> Espacio
8. SistemaEstacionamiento-> obtenerDetalle() -> Ticket

ObjetoEmisor	metodo()	ObjetoReceptor
SistemaEstacionamineto->	ocupar(vehiculo) ->	Espacio
SistemaEstacionamineto->	getHoraEntrada()->	Vehiculo
SistemaEstacionamineto->	new Ticket() ->	Ticket
Ticket->	getPlacas() ->	Vehiculo
Ticket->	getNumero() ->	Espacio
Ticket->	liberar() ->	Espacio
Ticket->	estaLibre ->	Espacio
SistemaEstacionamiento->	obtenerDetalle() ->	Ticket

Capturas de ejecución



```
cetza@AACC2007BS MINGW64 ~/OneDrive/TSWC2/POO/POO_PROYECTO
$  /usr/bin/env C:\\Program\ Files\\Java\\jdk-21\\bin\\java.exe -XX:+ShowCodeDetailsInExceptionMessages -cp C:\\Users\\cetza\\AppData\\Roaming\\Code\\User\\workspaceStor
age\\00390b3a4c70a4ceb772b841613203fa\\redhat.java\\jdt_ws\\POO_PROYECTO_c826cc91\\bin Main
 ========RESUMEN==========
Total entradas: 3
Total salidas: 2
Estado del espacio 1: Libre
Estado del espacio 2: Ocupado
Estado del espacio 3: Libre


========DETALLES DEL VEHICULO==========
Folio: T-1.
PLaca: MYT45-45.
Espacio: 1.
Hora de salida: 2:00 am.
Estado: Cerrado.
Situación: Vehiculo recogido


Folio: T-2.
PLaca: SDT02-78.
Espacio: 2.
Hora de salida: N/A.
Estado: Abierto.
Situación: Vehiculo estacionado


Folio: T-3.
PLaca: MKOI-95.
Espacio: 3.
Hora de salida: 9:00 am.
Estado: Cerrado.
Situación: Vehiculo recogido

cetza@AACC2007BS MINGW64 ~/OneDrive/TSWC2/POO/POO_PROYECTO
```